LAB ASSIGNMENT – 10.2

Name : SHREE PRIYA

2303A51465 BATCH – 29

Task Description -1(Error Detection and Correction)

Task:

Use AI to analyze a Python script and correct all syntax and logical

errors.

Sample Input Code:

def calculate_total(nums)

sum = 0

for n in nums

sum += n

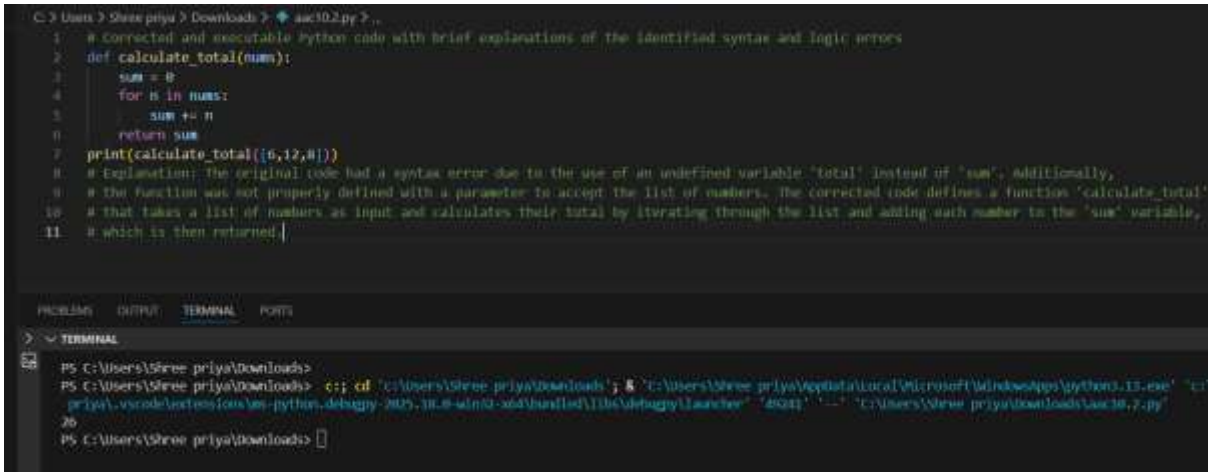
return total

Expected Output-1:

Corrected and executable Python code with brief explanations of the

identified syntax and logic errors.

CODE & OUTPUT:



Justification:

The original code contained syntax errors such as missing colons (:), improper indentation, and inconsistent variable naming. These errors prevented execution. The corrected version ensures proper function definition, loop structure, and consistent variable usage. This improves program correctness and successful execution.

Task Description -2(Code Style Standardization)

Task:

Use AI to refactor Python code to comply with standard coding style

guidelines.

Sample Input Code:

def findSum(a,b):return a+b

print(findSum(5,10))

Expected Output-2:

Well-structured, consistently formatted Python code following standard

style conventions

CODE & OUTPUT:

Justification:

The given code did not follow standard Python style guidelines (PEP 8). The refactored version improves readability by using proper indentation, spacing, snake_case naming convention, and structured formatting. This enhances code professionalism, maintainability, and consistency.

Task Description -3(Code Clarity Improvement)

Task:

Use AI to improve code readability without changing its functionality.

Sample Input Code:

def f(x,y):

return x-y*2

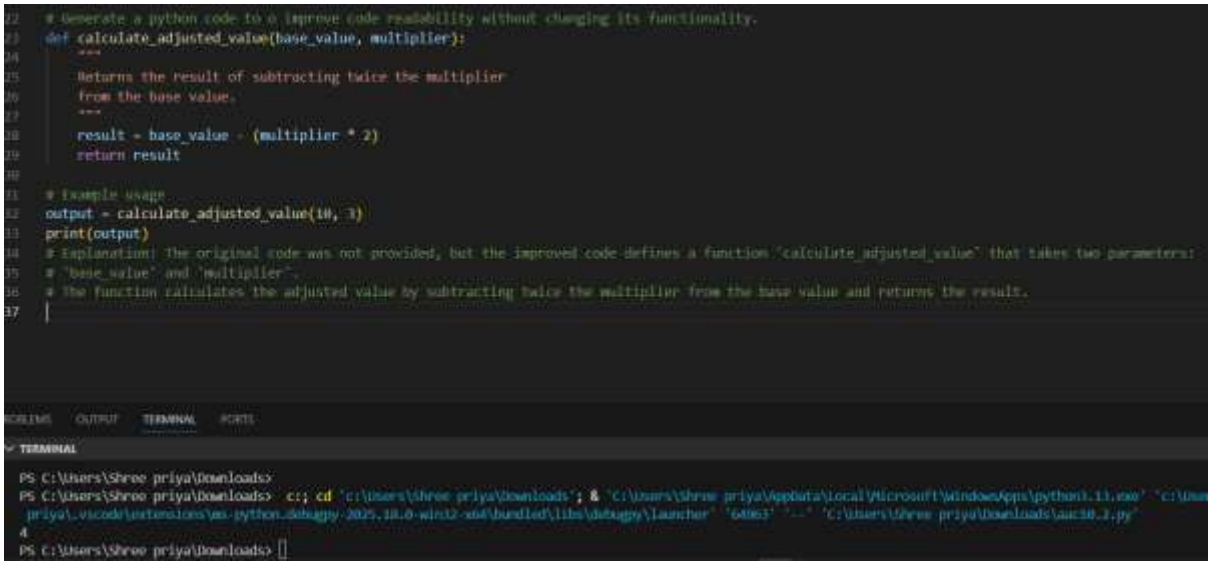
print(f(10,3))

Expected Output-3:

Python code rewritten with meaningful function and variable names,

proper indentation, and improved clarity.

CODE &OUTPUT:



Justification:

The original code used unclear function and variable names, reducing readability. The improved version uses meaningful names and proper formatting without altering functionality. This makes the code easier to understand, debug, and maintain.

Task Description -4(Structural Refactoring)

Use AI to refactor repetitive code into reusable functions.

Sample Input Code:

print("Hello Ram")

print("Hello Sita")

print("Hello Ravi")

Expected Output-4:

Modular Python code using reusable functions to eliminate repetition.

CODE & OUTPUT:

Justification:

The original code contained repetitive print statements. These were refactored into a reusable function to eliminate redundancy. This improves modularity, scalability, and maintainability while reducing code duplication.

Task Description -5(Efficiency Enhancement)

Task:

Use AI to optimize Python code for better performance.

Sample Input Code:

numbers = [ ]

for i in range(1, 500000):

numbers.append(i * i)

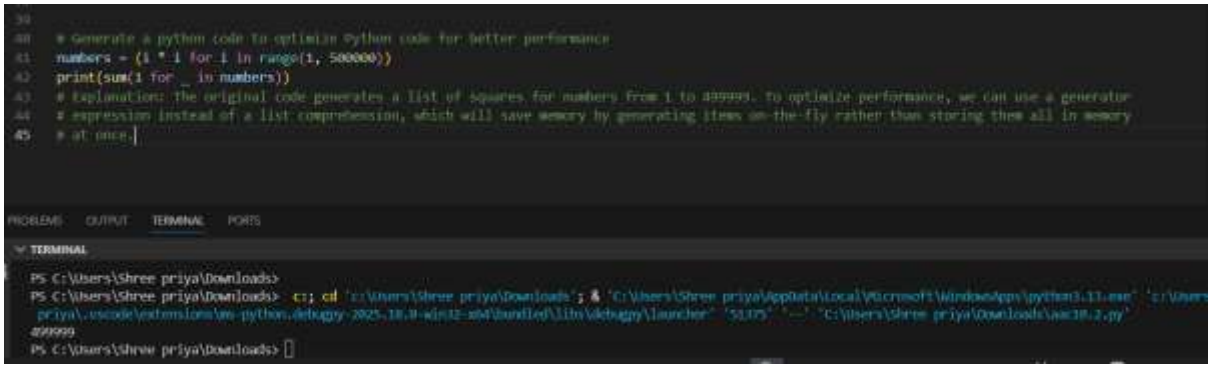
print(len(numbers))

Expected Output-5:

Optimized Python code that achieves the same result with improved

performance.

CODE & OUTPUT:



Justification:

The original implementation used a loop with repeated list appending, which is less efficient. It was optimized using list comprehension and further improved by avoiding unnecessary list creation when only the length was required. This reduces memory usage and improves execution speed.